



Fundamentals of Accelerated Data Science

NVIDIA Deep Learning Institute

“
An Introduction to Data Science
with a Focus on Speed and
Efficiency

NVIDIA Deep Learning Institute

”

What This Course Will Cover

An introduction to data science with a focus on speed and efficiency

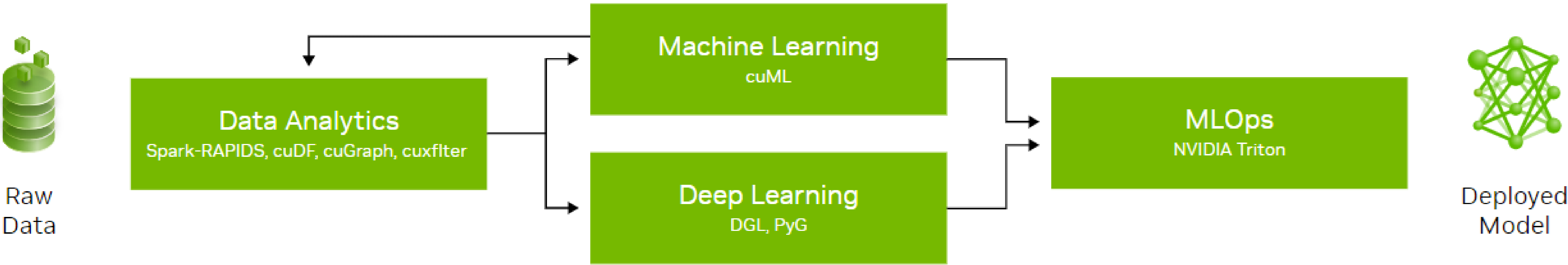
Learning objectives:

- Overview of data science
- Demonstrations of accelerated data science pipelines
- How acceleration is achieved and its implications

This workshop will not cover:

- Neural networks (accelerated by cuDNN)
- Statistical analysis

	Course Outline
Task 1	Data science overview Data manipulation
Task 2	Machine learning
Task 3	Graph analytics
Coding Assessment	Biodefense





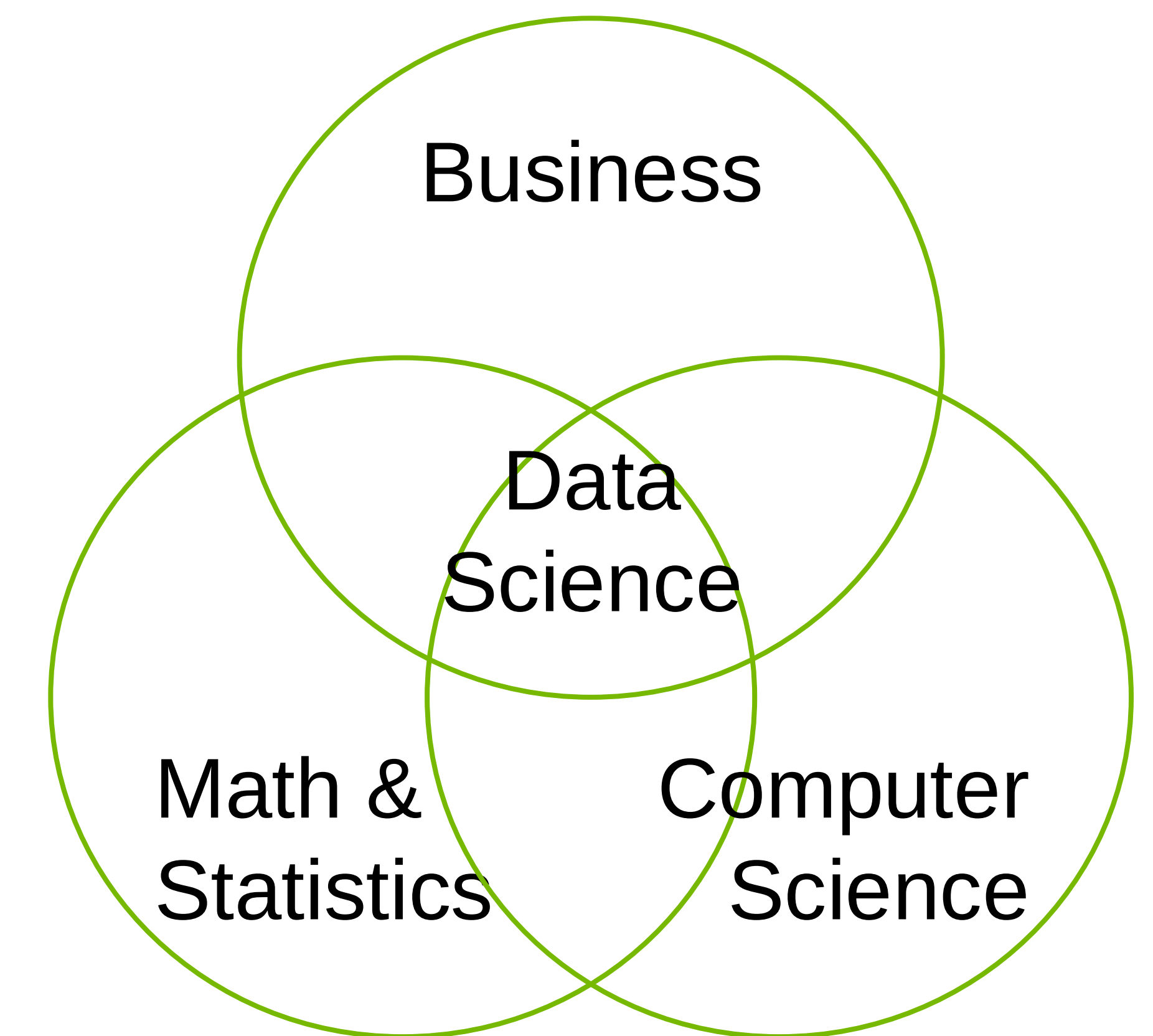
Section Agenda

- Data Science Fundamentals
- RAPIDS
- Accelerating and Scaling Data Manipulation
- Hands-On Lab

Data Science Overview

The study of data to extract meaningful insights

- Data science is a multidisciplinary approach that combines principles and practices from the fields of mathematics, statistics, artificial intelligence, and computer engineering to analyze large amounts of data.
- More than just quantitative analysis - it combines knowledge in using tools, methods, and technology to generate meaning from data
- Helps to ask and answer questions like:
 - Descriptive Analysis (what happened)
 - Diagnostic Analysis (why it happened)
 - Predictive Analysis (what will happen)
 - Prescriptive Analysis (what can be done with the results)



Data Science Is the Key to Modern Business

Enables data-driven decision making



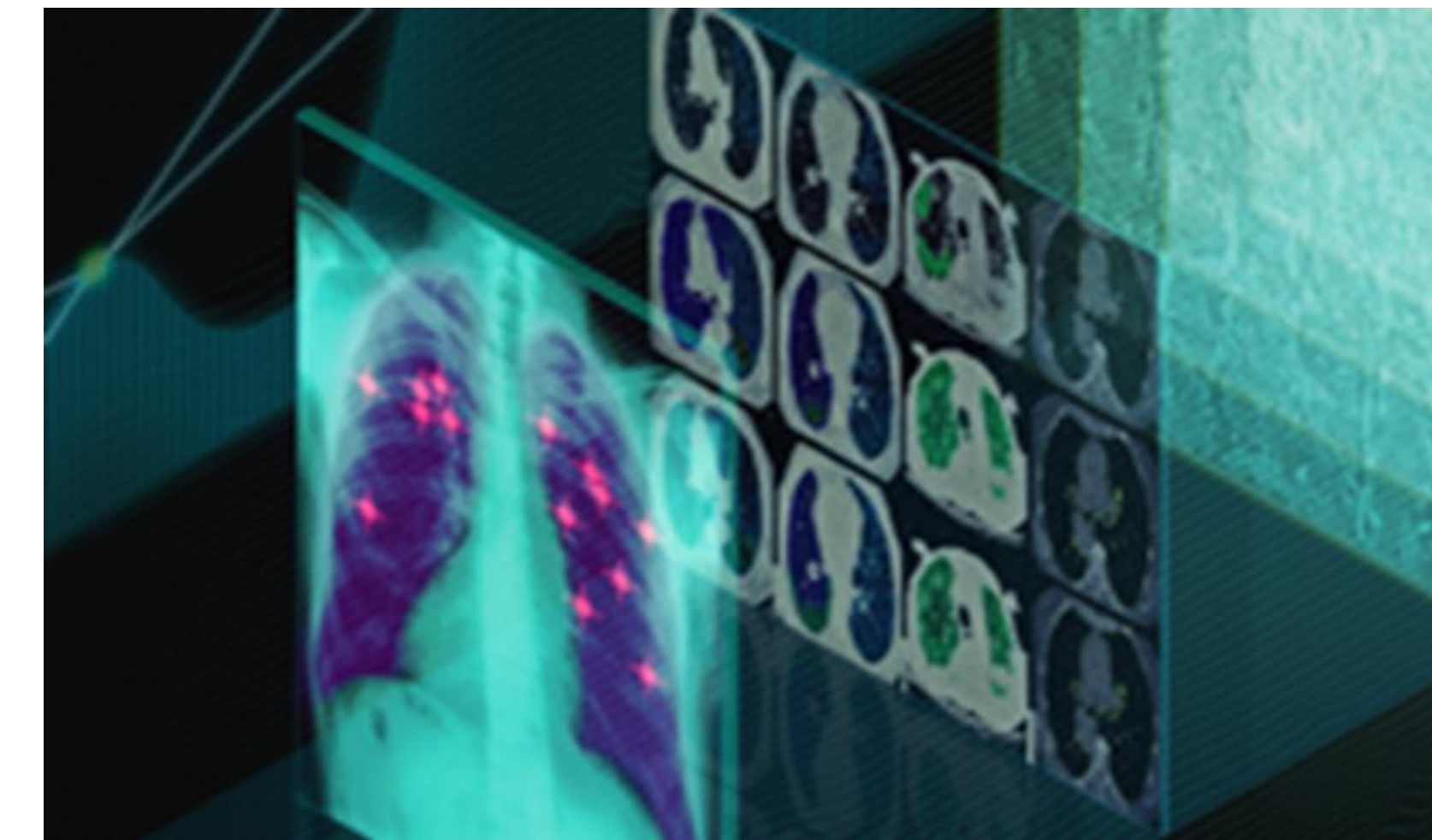
Retail

Supply Chain & Inventory Management
Price Management / Markdown Optimization
Promotion Prioritization And Ad Targeting



Telecom

Detect Network/Security Anomalies
Forecasting Network Performance
Network Resource Optimization (SON)



Healthcare

Improve Clinical Care
Drive Operational Efficiency
Speed Up Drug Discovery



Manufacturing

Remaining Useful Life Estimation
Failure Prediction
Demand Forecasting



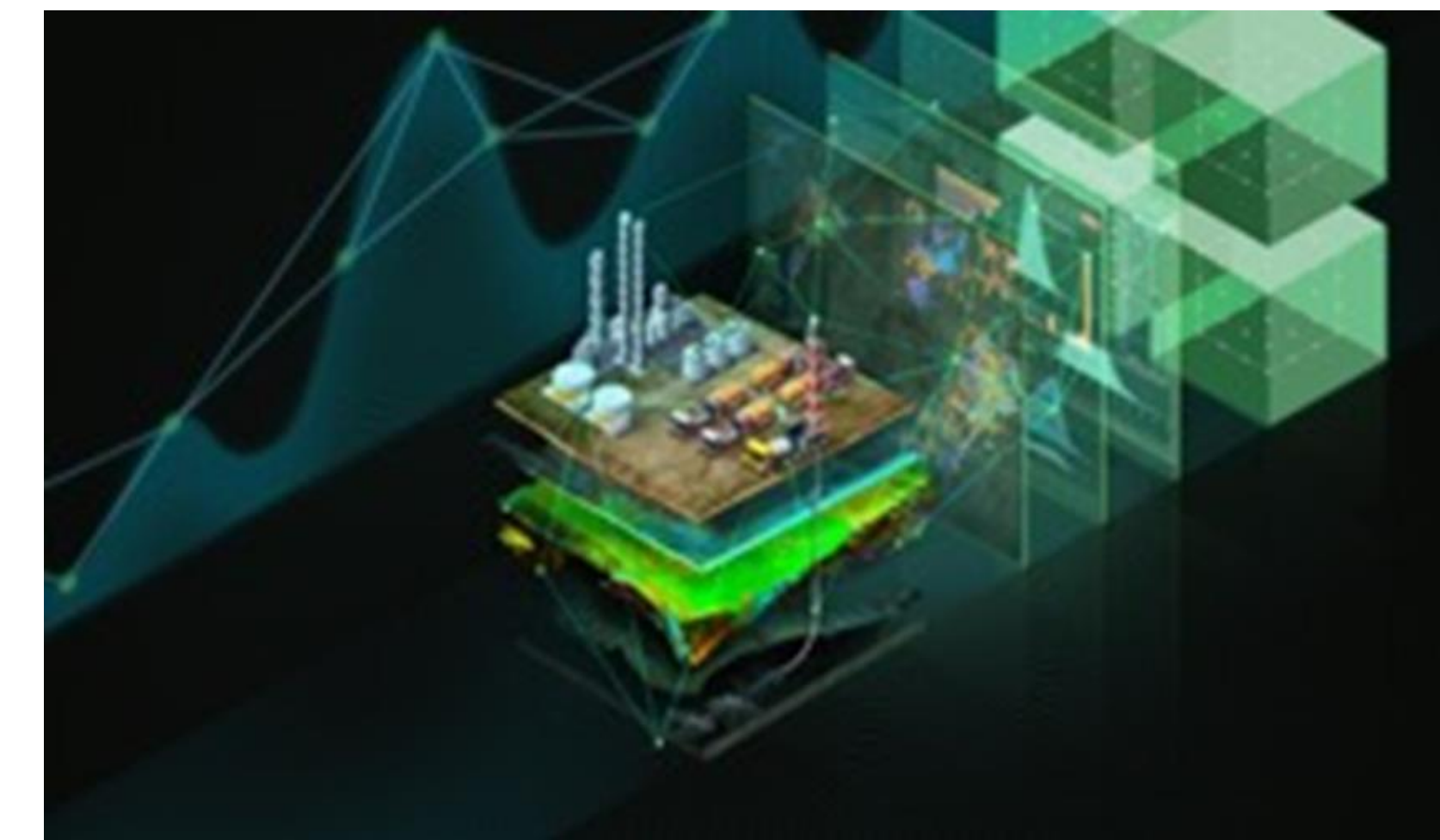
Financial Services

Claim Fraud
Customer Service Chatbots/Routing
Risk Evaluation



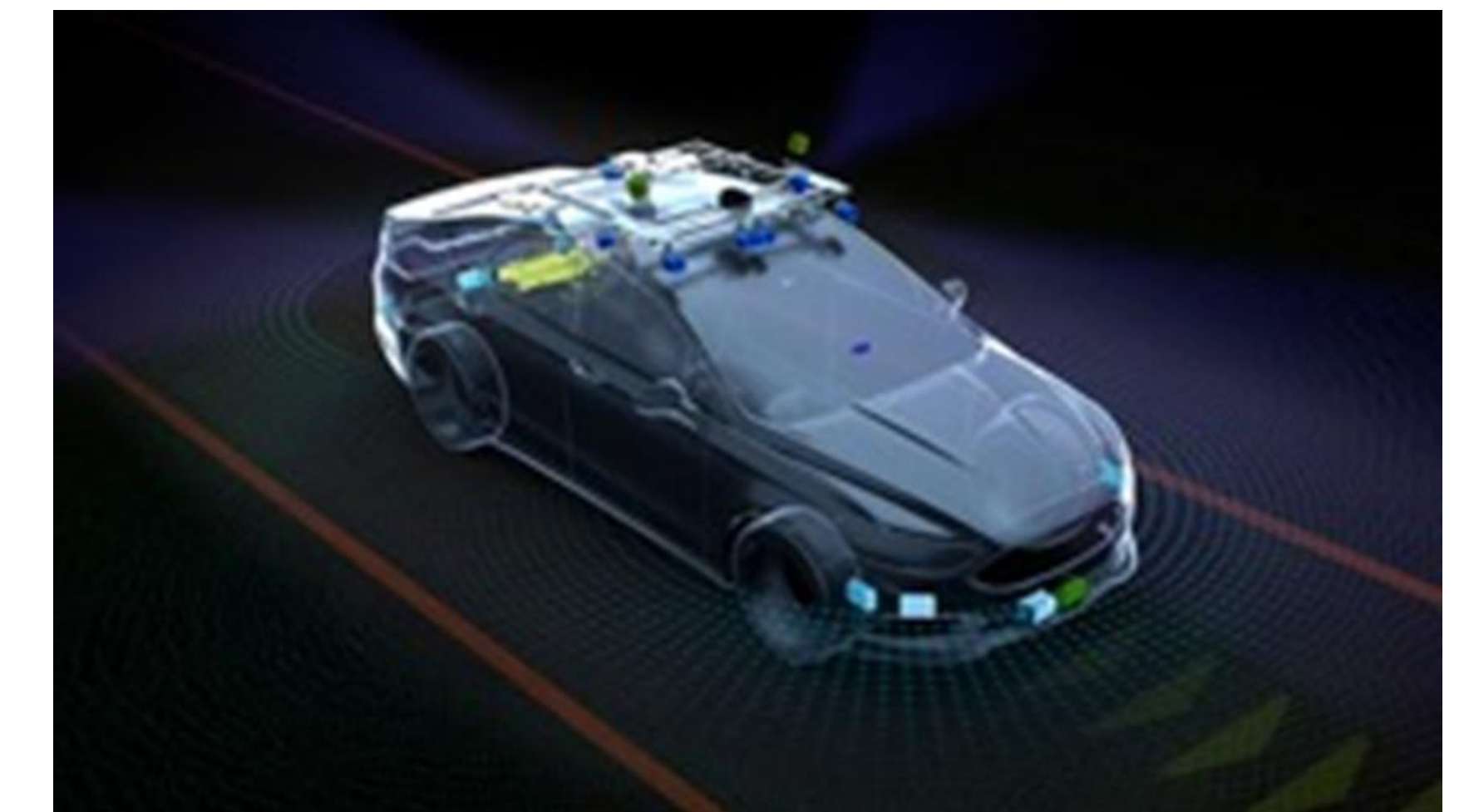
Consumer Internet

Ad Personalization
Click Through Rate Optimization
Churn Reduction



Oil & Gas

Sensor Data Tag Mapping
Anomaly Detection
Robust Fault Prediction



Automotive

Personalization & Intelligent Customer Interactions
Connected Vehicle Predictive Maintenance
Forecasting, Demand, & Capacity Planning

Key Industry Use Cases

Forecasting, fraud detection, recommender systems, and more...

FORECASTING



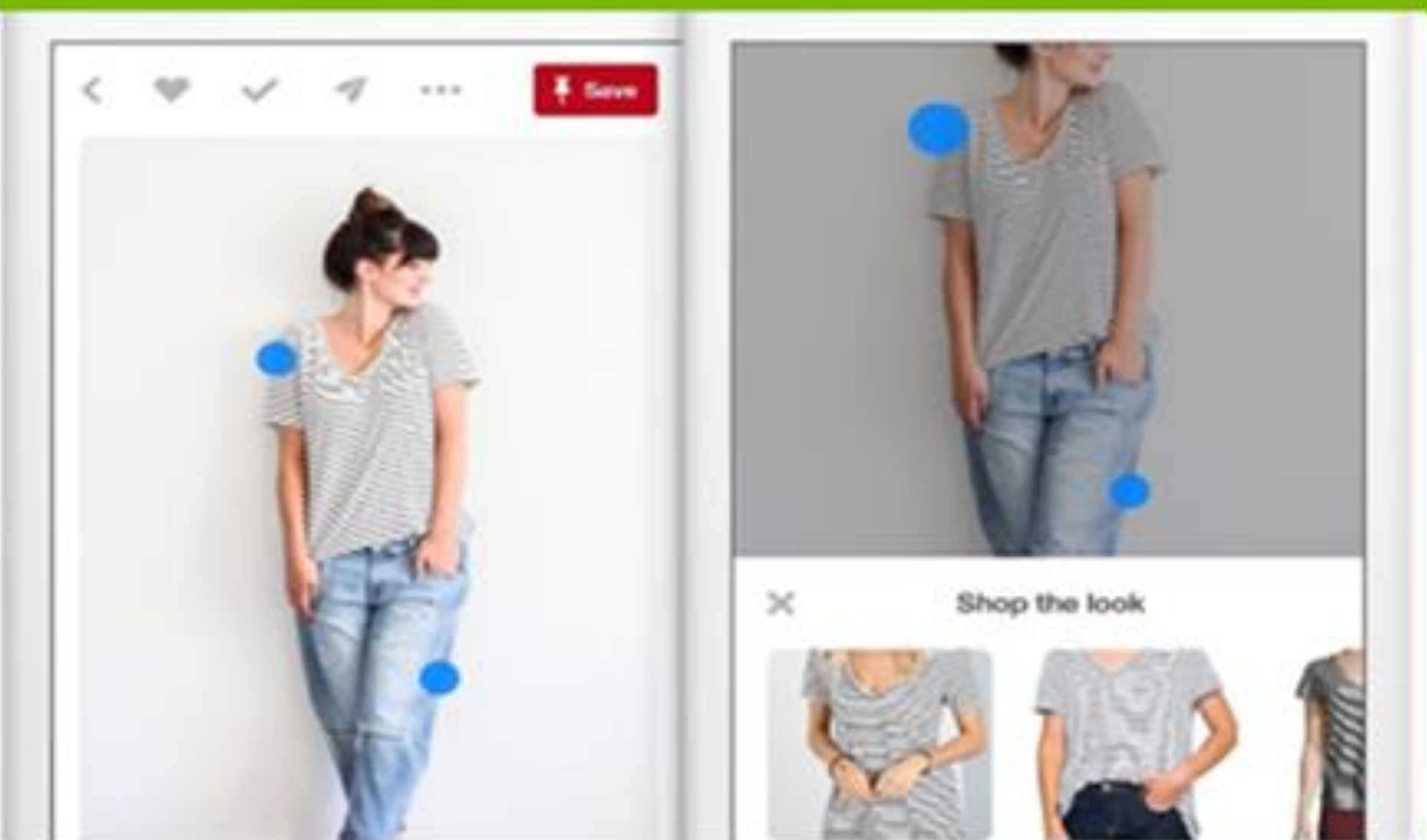
Data science significantly increases the efficiency and accuracy of forecasting, directly contributing to bottom line growth.

FRAUD DETECTION



Data science dramatically improves fraud detection, saving money for organizations and customers alike.

RECOMMENDER SYSTEMS



Data science has revolutionized the connected experience, delighting users with personalized experiences.

Data Science Is More Than Machine Learning

Non-analytical tasks related to data science are equally important

Analytical tasks include:

- Machine learning model development
- Measuring key metrics
- Statistical analysis
- Data visualizations

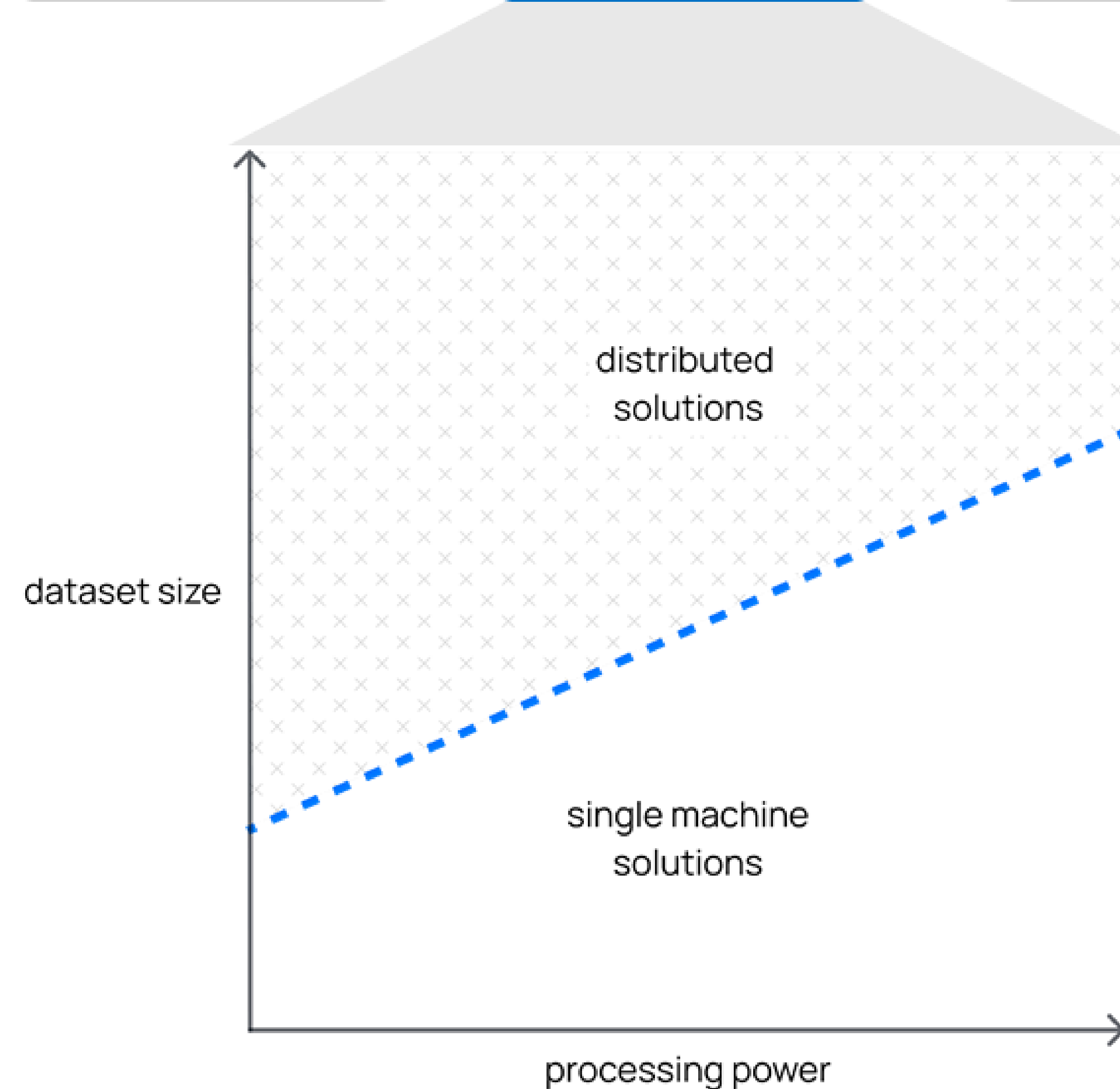
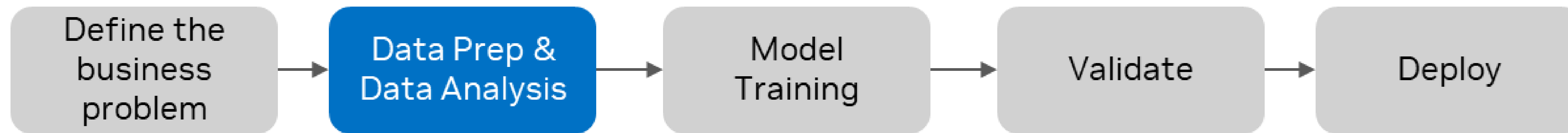
Non-analytical tasks (**upstream**) include:

- **Extract, transform, and load (ETL)**
 - Data collection
 - Data cleaning and preprocessing
 - Data modeling
- **Data engineering**



Data Analytics is a Bottleneck for AI Pipelines

40% of data scientist's time



More powerful compute is enabling large(r) scale data analytics on single machine.

Challenges in Data Science Today

What needs to be solved to empower data scientists



INCREASING BIG DATA USE CASES

Data sets are continuing to dramatically increase in size

Multitude of sources

Different data formats with varying quality



SLOW CPU PROCESSING

End of Moore's law, CPUs aren't getting faster

Many popular data science tools are CPU-only

CPUs are not designed to parallelize process well



COMPLEX SOFTWARE MANAGEMENT

Time consuming to install software

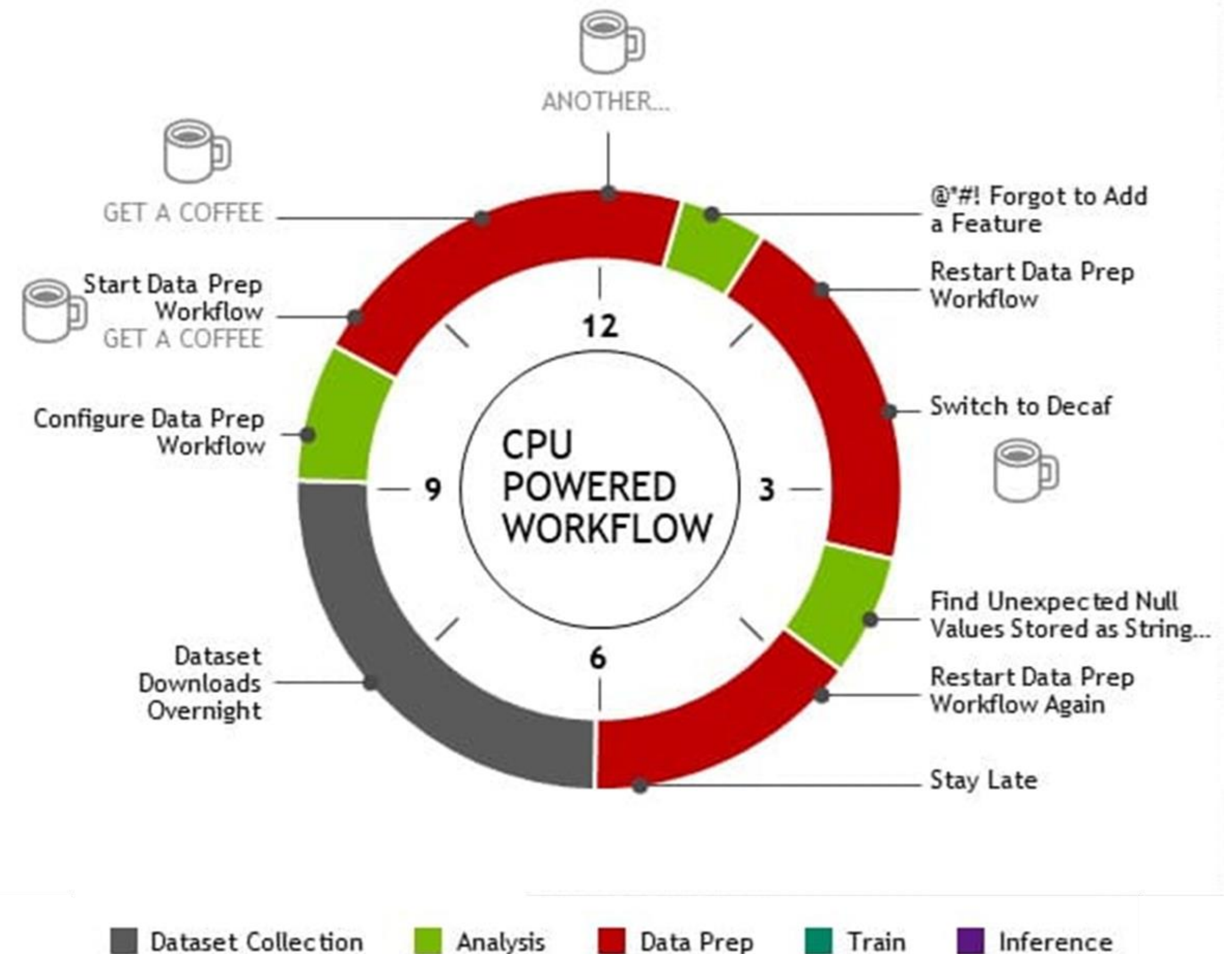
Nearly impossible to manage all version conflicts

Updates often break other software

Challenges in Data Science Today

The right solution enables data scientists to focus on delivering value

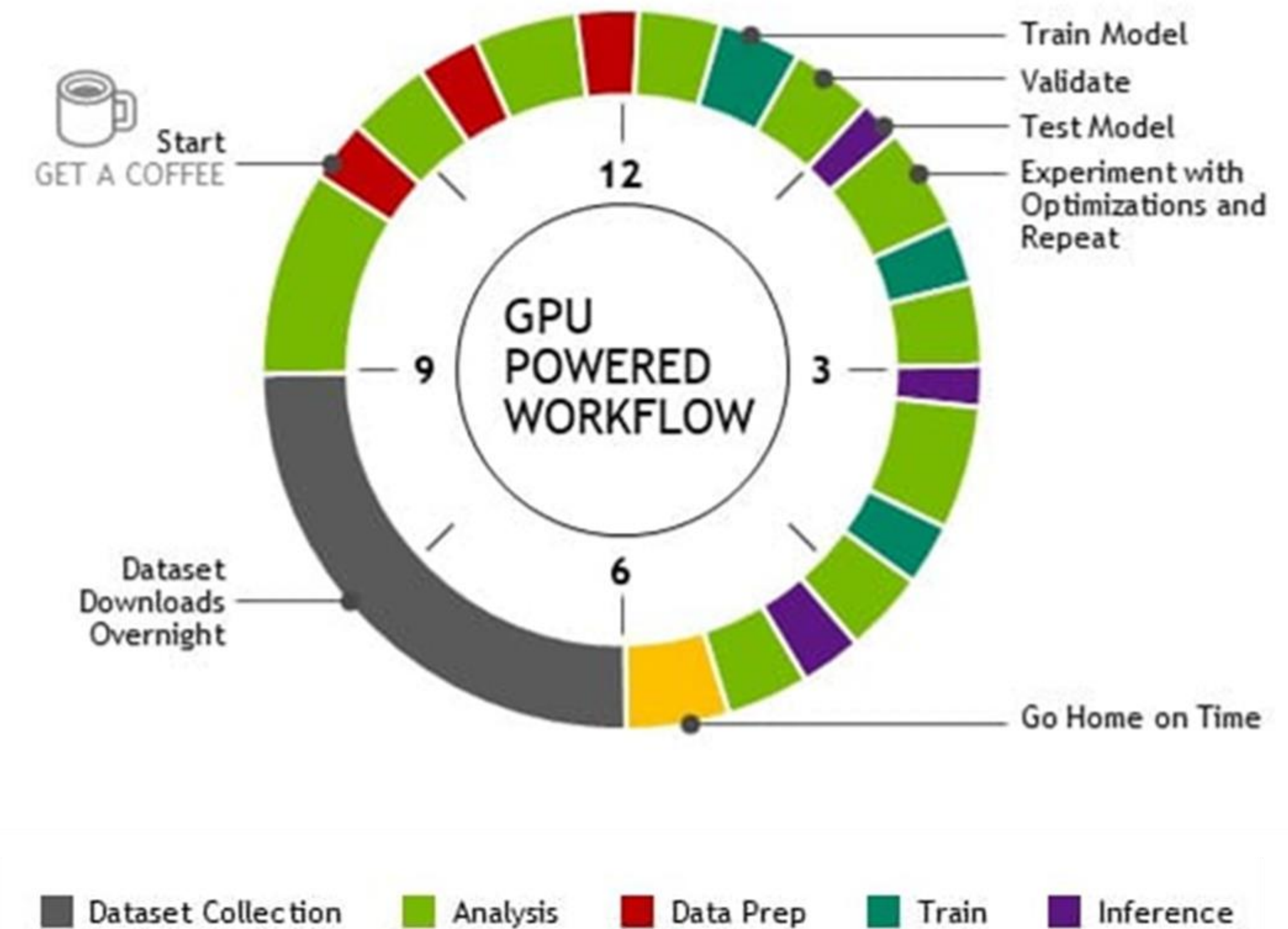
- Data preparation is a time-consuming process
- Data science requires **exploration** and **experimentation** to:
 - Deal with uncertainties
 - Discover and adapt to unexpected insights
 - Validate, refine, and optimize results
- Size, speed, and complexity of data continues to grow
- Amount of data that can be analyzed is limited due to resource constraints
- Existing solutions are complex when talent is in short supply



Challenges in Data Science Today

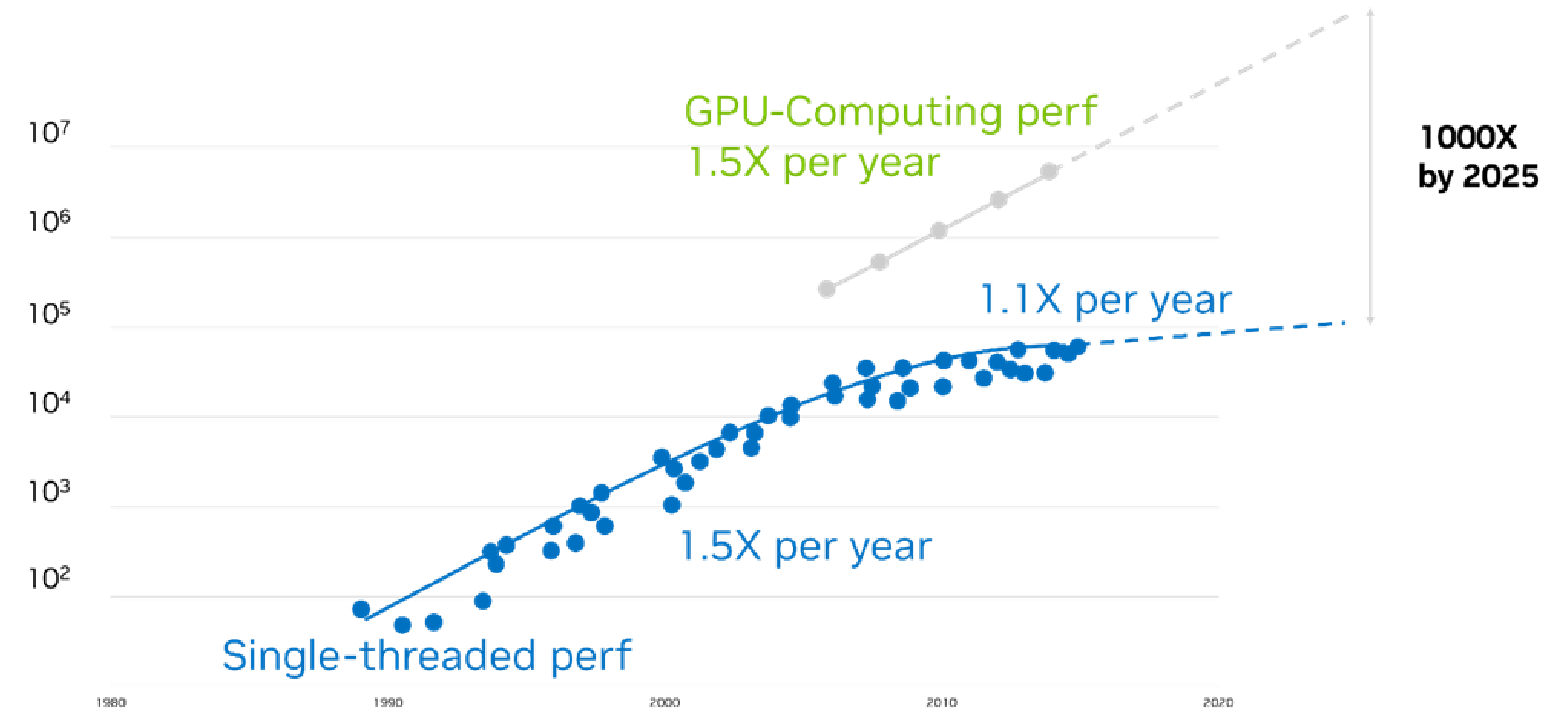
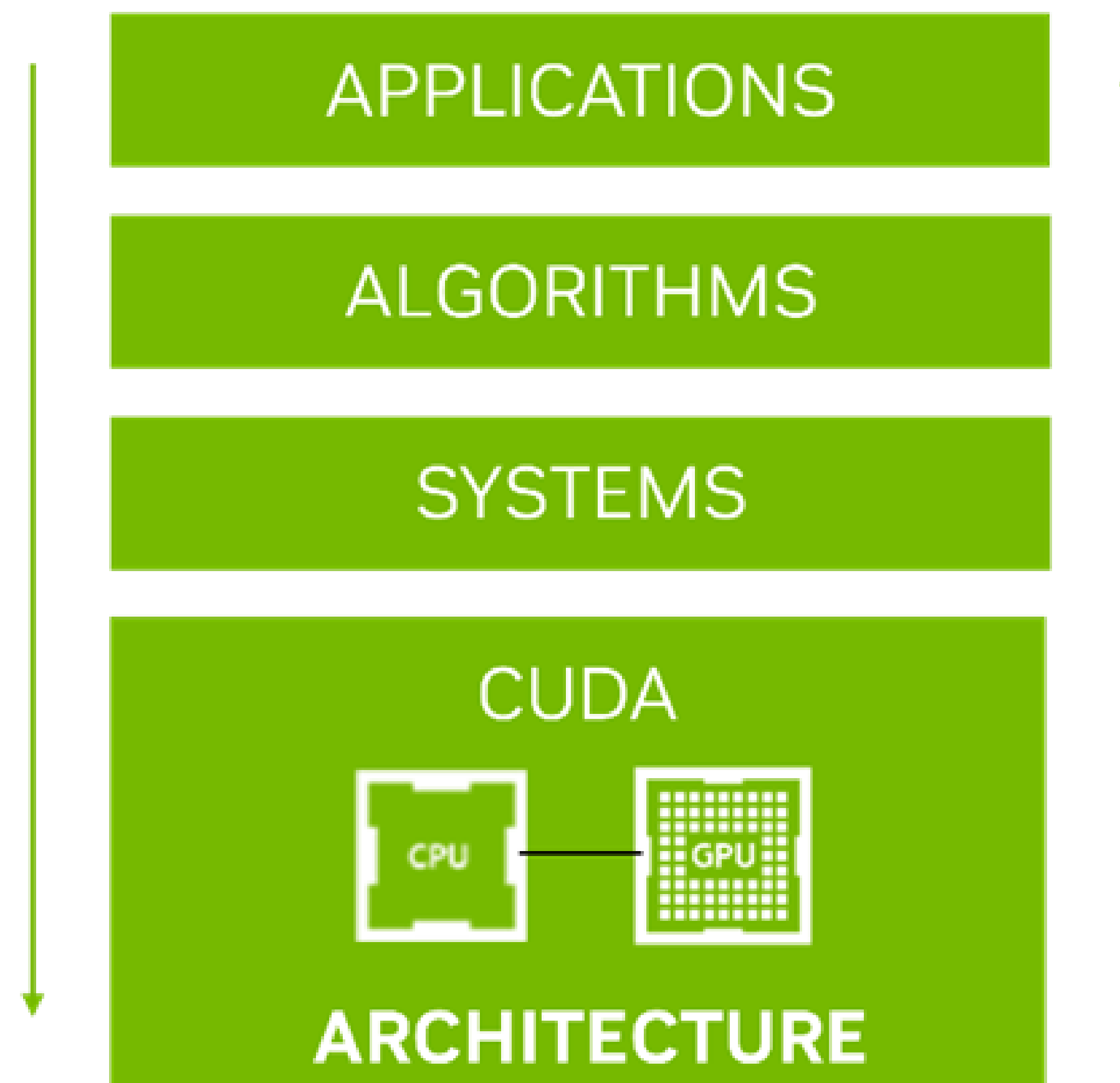
The right solution enables data scientists to focus on delivering value

- Faster data processing is key
- Benefits of accelerated data science workflows:
 - Higher productivity
 - More explorations and iterations can improve model accuracy and optimize inference performance
 - Lower time to insight and total cost of ownership



Rise of GPU Computing

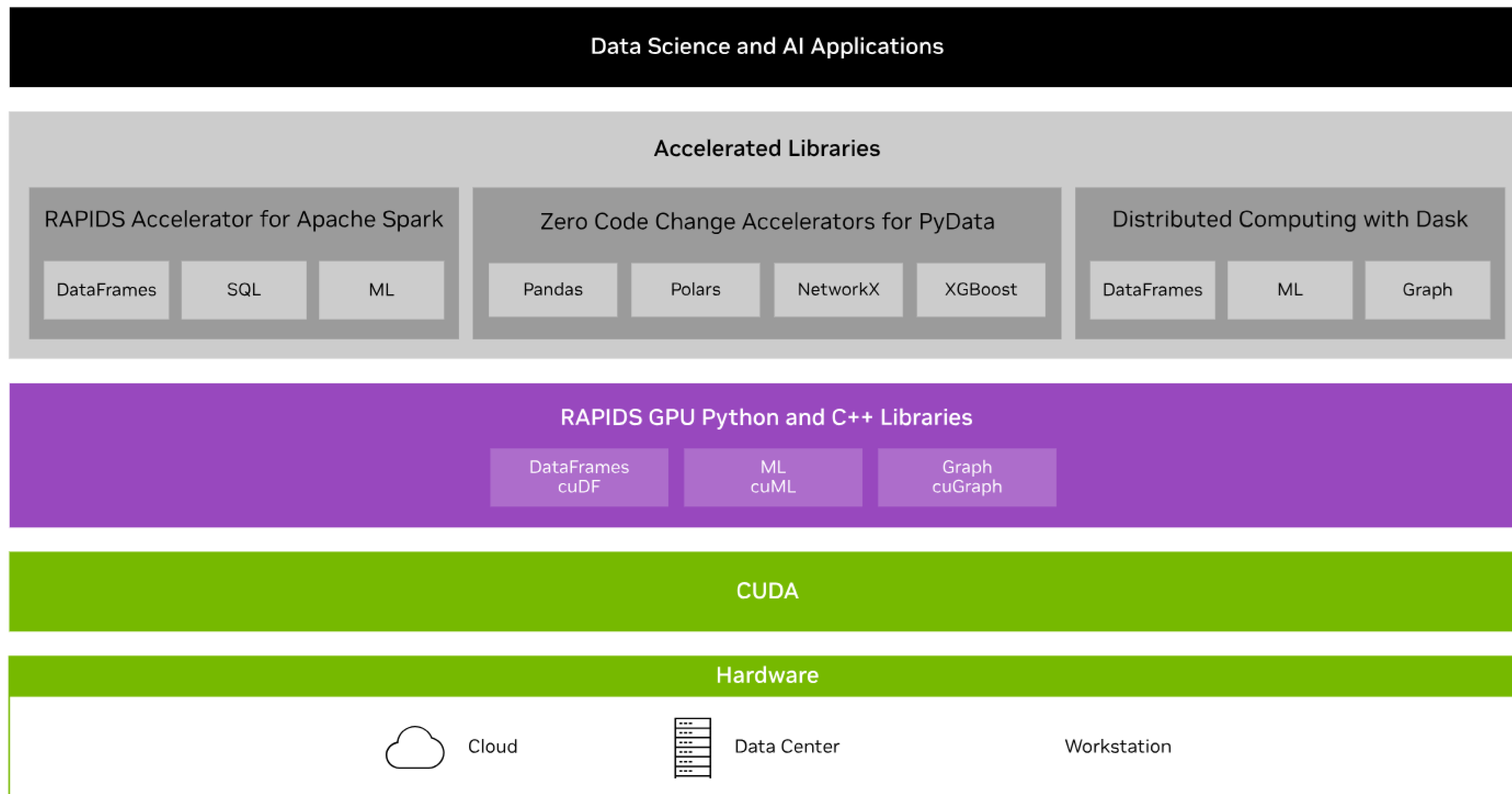
Leverages parallel processing to handle large datasets



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten New plot and data collected for 2010-2015 by K. Rupp

RAPIDS Transforms Data Science

An optimized hardware-to-software stack for the entire data science pipeline



Pillars of Data Science Performance

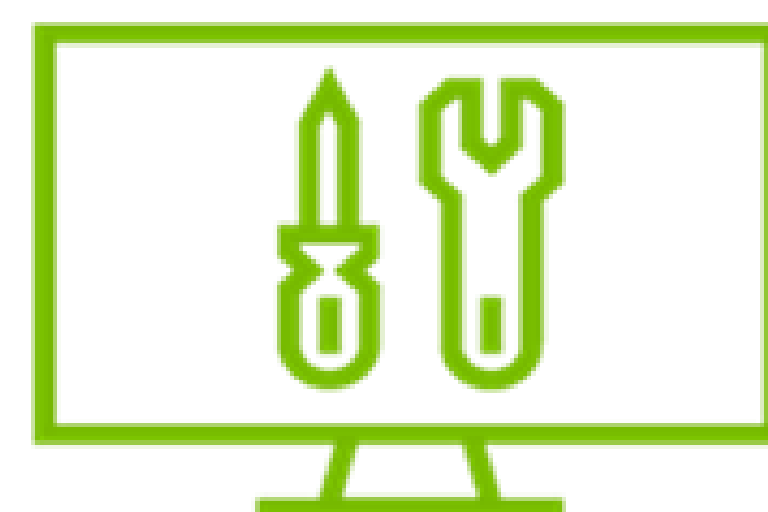
Empower data scientists to do their best work

Enable acceleration while minimizing friction related to infrastructure



Fast Execution Time

Data science is all about speed to results. RAPIDS leverages [NVIDIA CUDA®](#) under the hood to accelerate your workflows by running the entire data science training pipeline on GPUs. This reduces training time and the frequency of model deployment from days to minutes.



Use the Same Tools

By hiding the complexities of working with the GPU and even the behind-the-scenes communication protocols within the data center architecture, RAPIDS creates a simple way to get data science done. As more data scientists use Python and other high-level languages, providing acceleration without code change is essential to rapidly improving development time.

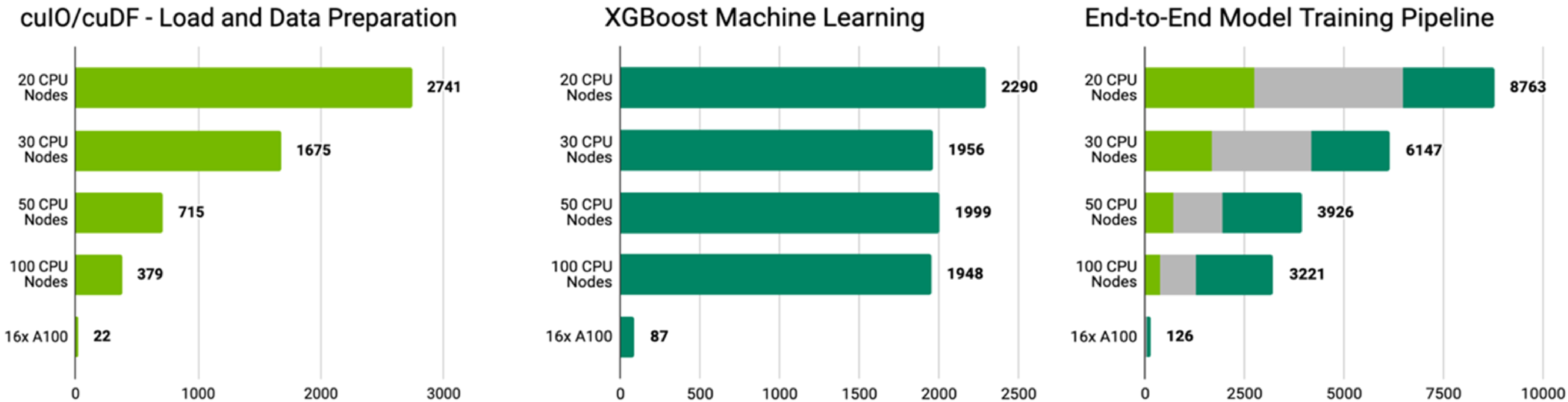


Run Anywhere at Scale

RAPIDS can be run anywhere—cloud or on-prem. You can easily scale from a workstation to multi-GPU servers to multi-node clusters, as well as deploy it in production with Dask, Spark, MLFlow, and Kubernetes.

Proven Faster for Data Science

Realize efficiency and scalability with multi-GPU for massive datasets



Benchmark

200GB CSV dataset; Data preparation includes joins, variable transformations.

CPU Cluster Configuration

CPU nodes (61 GiB of memory, 8 vCPUs, 64-bit platform), Apache Spark

DGX Cluster Configuration

5x DGX-1 on InfiniBand network

Time in seconds — Shorter is better

■ cuIO / cuDF (Load and Data Preparation) ■ Data Conversion ■ XGBoost

DataFrame

Data structure is distinguished by type, industry, modality, and the tools used to manipulate

Structured (predefined format/schema):

- Tables
- Spreadsheets
- CSVs
- JSONs (semi-structured)
- Relational databases

Unstructured (doesn't follow a specific data model or schema):

- Texts
- Images
- Audios
- Genes
- Graphs

DataFrame

Data structure is distinguished by type, industry, modality, and the tools used to manipulate

Structured (predefined format/schema):

- Tables
- Spreadsheets
- CSVs
- JSONs (semi-structured)
- Relational databases

	Column 1	Column 2	Column 3
Row 1	1	2	3
Row 2	2	3	4
Row 3	3	4	5

Unstructured (doesn't follow a specific data model or schema):

- Texts
- Images
- Audios
- Genes
- Graphs

DataFrame

Data structure is distinguished by type, industry, modality, and the tools used to manipulate

Structured (predefined format/schema):

- Tables
- Spreadsheets
- CSVs
- JSONs (semi-structured)
- Relational databases

	Column 1	Column 2	Column 3
Row 1	1	2	3
Row 2	2	3	4
Row 3	3	4	5

Unstructured (doesn't follow a specific data model or schema):

- Texts
- Images
- Audios
- Genes
- Graphs

DataFrame

Data structure is distinguished by type, industry, modality, and the tools used to manipulate

Structured (predefined format/schema):

- Tables
- Spreadsheets
- CSVs
- JSONs (semi-structured)
- Relational databases

$$\begin{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \end{bmatrix}$$

Unstructured (doesn't follow a specific data model or schema):

- Texts - sequence of characters, words, and sentences
- Images - 2D array of pixels
- Audios - sequence of amplitude and frequency (waveform)
- Genes - sequence of proteins or graphs
- Graphs

Structure will play a role in acceleration, when we can represent data as vectors

Basic Idea

Vectorized operations are typically much faster than equivalent loops or iterations

Problem: (Column 1 - Column 2) * Column 3:

Iterative:

1. $(1 - 2) \times 3 = -3$
2. $(2 - 3) \times 4 = -4$
3. $(3 - 4) \times 5 = -5$

	Column 1	Column 2	Column 3
Row 1	1	2	3
Row 2	2	3	4
Row 3	3	4	5

Vector operation (able to leverage parallel processing):

$$\left(\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} - \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} \right) \times \begin{bmatrix} 3 & 4 & 5 \end{bmatrix} = \begin{bmatrix} -3 \\ -4 \\ -5 \end{bmatrix}$$

Basic Idea

Total performance scales with both vector width and number of cores

Problem: (Column 1 - Column 2) * Column 3:

Iterative:

$$\begin{aligned} 1. & (1 - 2) \times 3 = -3 \\ 2. & (2 - 3) \times 4 = -4 \\ 3. & (3 - 4) \times 5 = -5 \end{aligned}$$

	Column 1	Column 2	Column 3
Row 1	1	2	3
Row 2	2	3	4
Row 3	3	4	5

Vector operation (able to leverage parallel processing):

$$\left(\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} - \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} \right) \times \begin{bmatrix} 3 & 4 & 5 \end{bmatrix} = \begin{bmatrix} -3 \\ -4 \\ -5 \end{bmatrix}$$

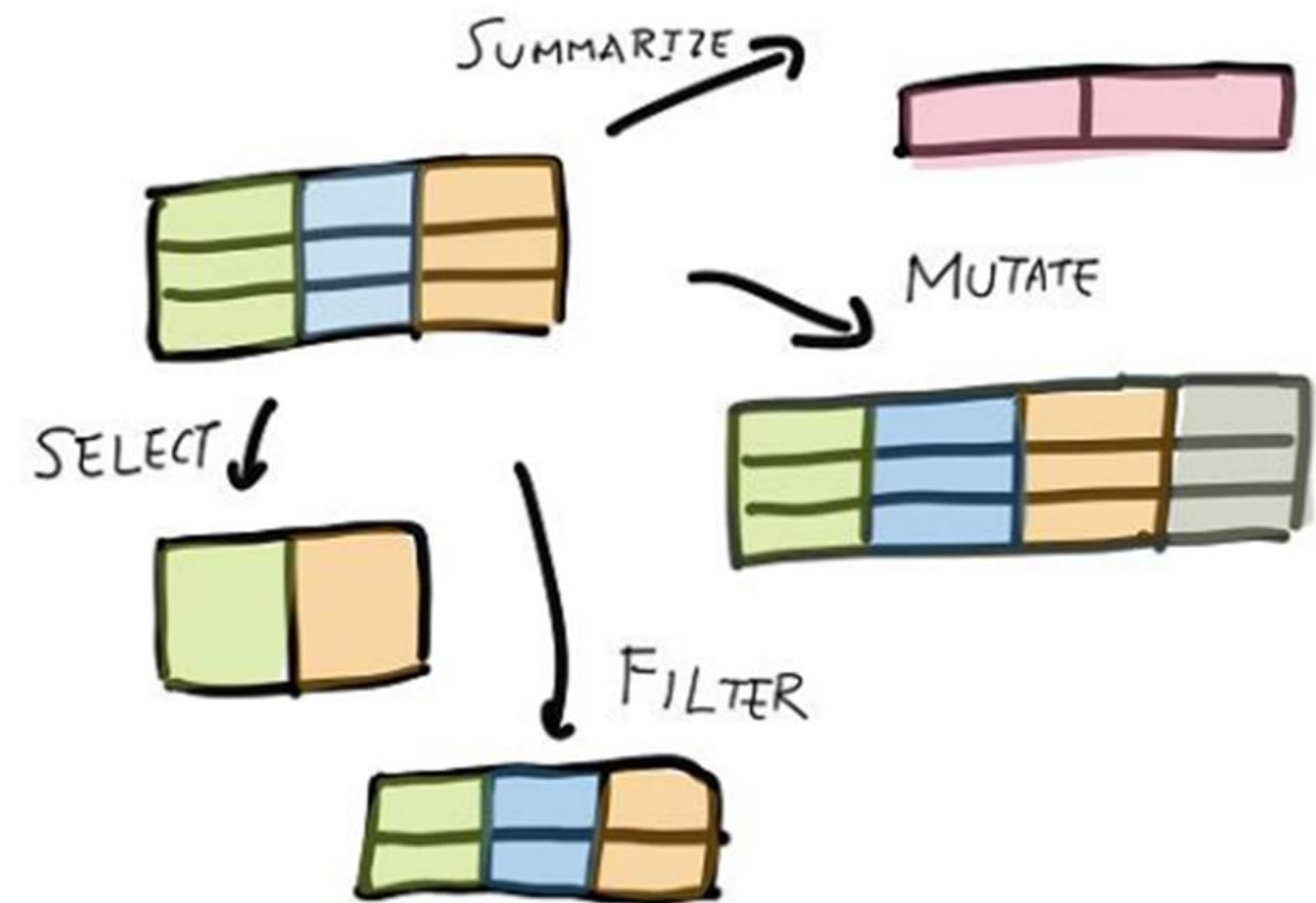
How Data Science Uses cuDF

GPU DataFrame library for loading, joining, aggregating, filtering, and otherwise manipulating data

- Python interface to CUDA C++ libraries for loading, joining, aggregating, filtering, and otherwise manipulating data using a DataFrame style API
- Includes:
 - Accelerated file readers for prevalent formats in data science
 - Powerful aggregation functions using GroupBy
 - Various string and list manipulation
 - Ability to write user-defined functions (UDFs), without needing to write CUDA code
 - JIT compilation of User-Defined Functions (UDFs) using Numba

Accelerates:

- ETL workflows
- Analytics Workflows
 - Faster statistical analysis
 - Quicker response from interactive visualization
- ML Workflows
 - Data preparation for downstream ML tasks
 - Remove feature engineering bottlenecks
 - Rapid experimentation



RAPIDS cuDF

Enabler for accelerated pandas and Polars

Several ways to use cuDF:

- Use cudf.pandas
 - Zero code change acceleration
 - Write code with the full flexibility of pandas, then load cudf.pandas to accelerate on the GPU, with automatic CPU fallback if needed
 - Third-party library compatible
 - One code path differing hardware
 - Accelerates the entire pandas ecosystem
- cuDF provides an in-memory, GPU-accelerated execution engine for Python users of the Polars Lazy API
- Write cuDF code using DataFrame style API

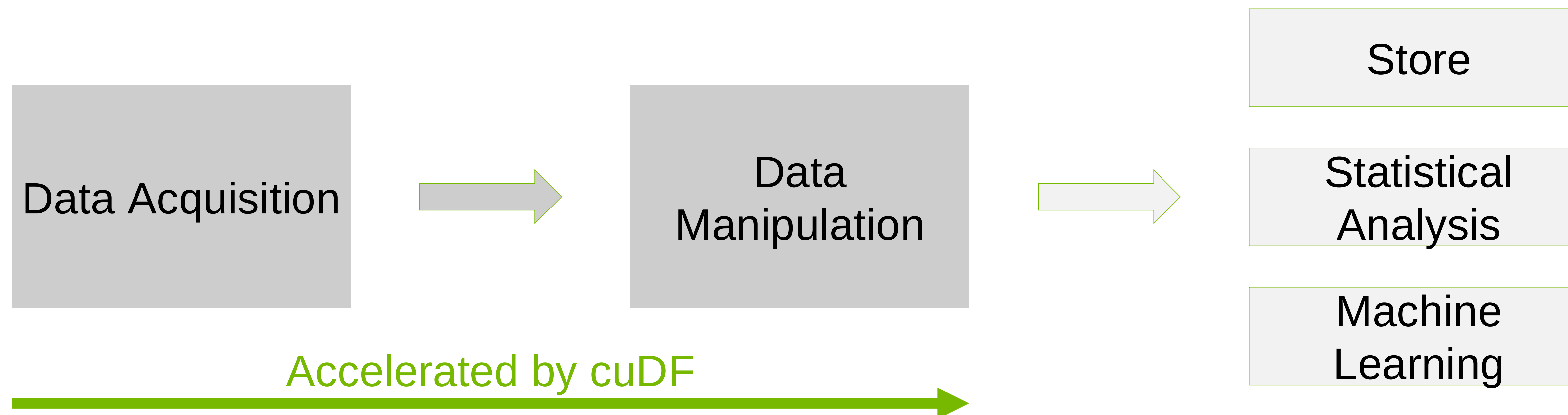


RAPIDS



Accelerating Data Acquisition and Data Manipulation

Data preparation for downstream tasks

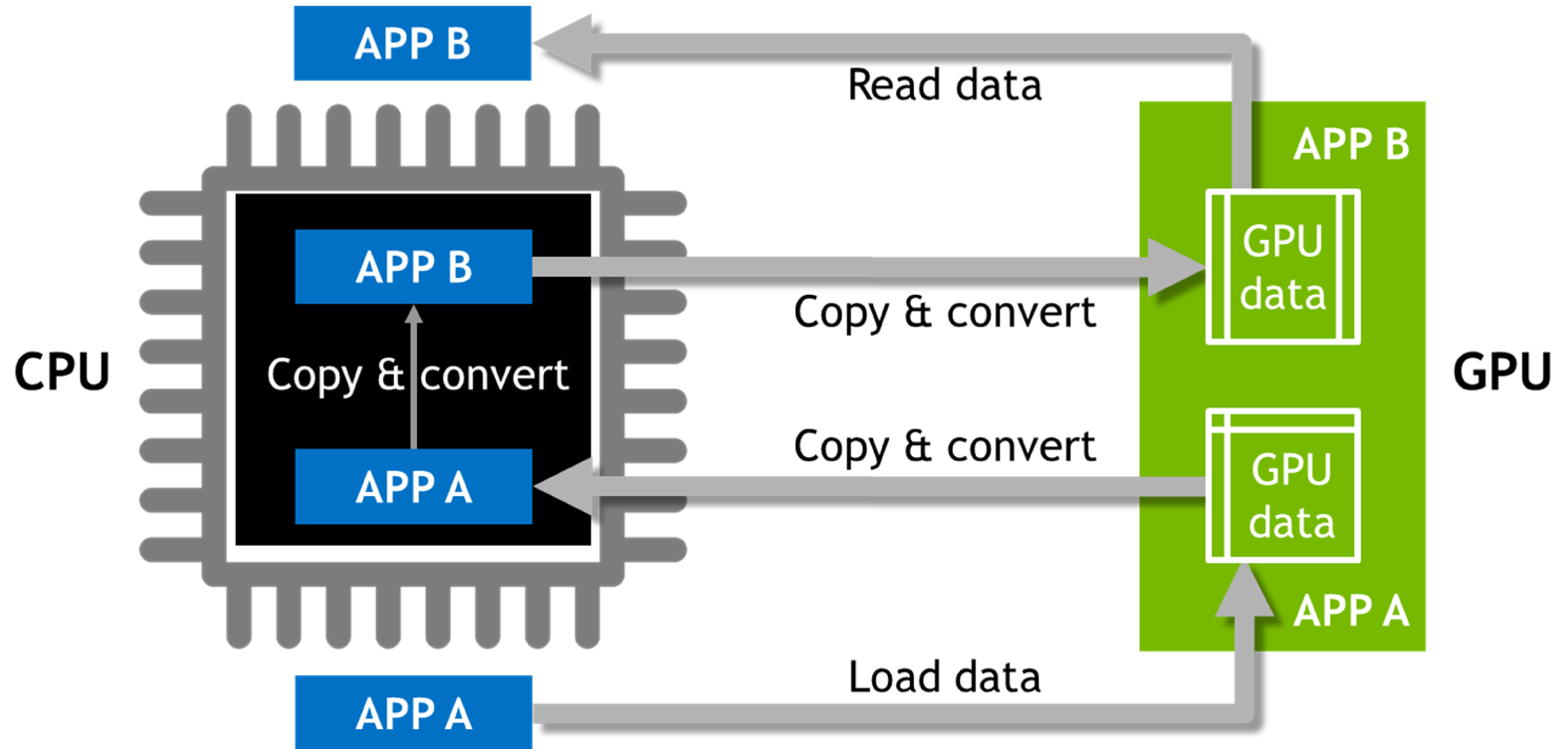


Tips:

- Memory management
 - Some vectorized operations may create temporary copies of data, which can be problematic for very large datasets
- Leverage parallel processing when possible (GPU)
 - Design operations to use vector operations
- Keeping data on the GPU as much as possible
 - Minimize copy/convert operations
- Diagnose and remove bottlenecks

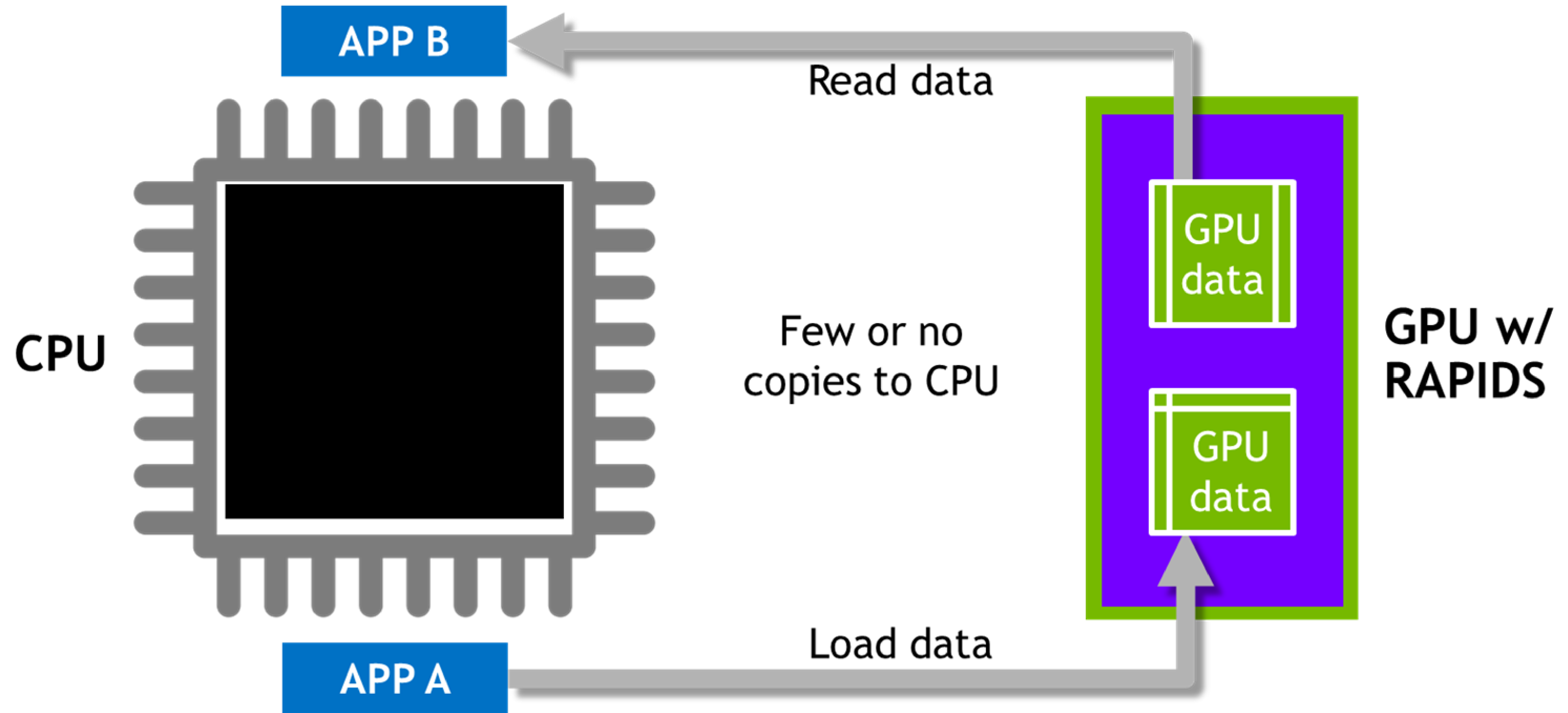
Traditional Model

Too much data movement and too many makeshift data formats require transformation



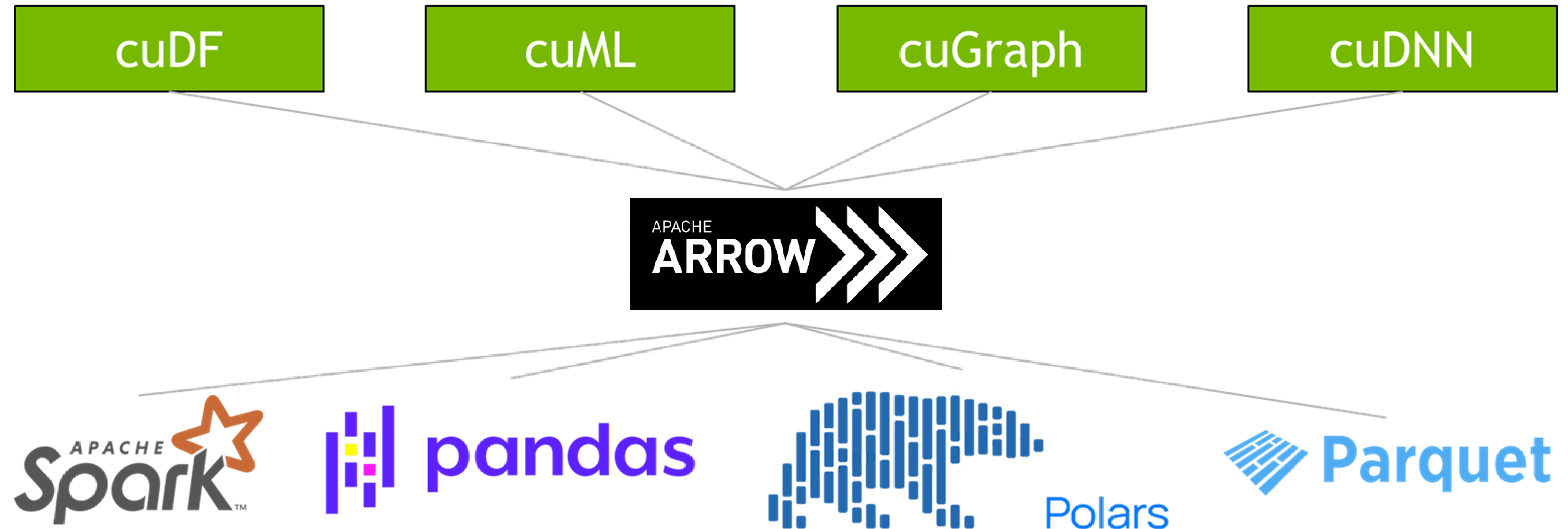
RAPIDS Model

Shared, standard data structures are essential to building an efficient workflow



One Format for Interoperability and Efficiency

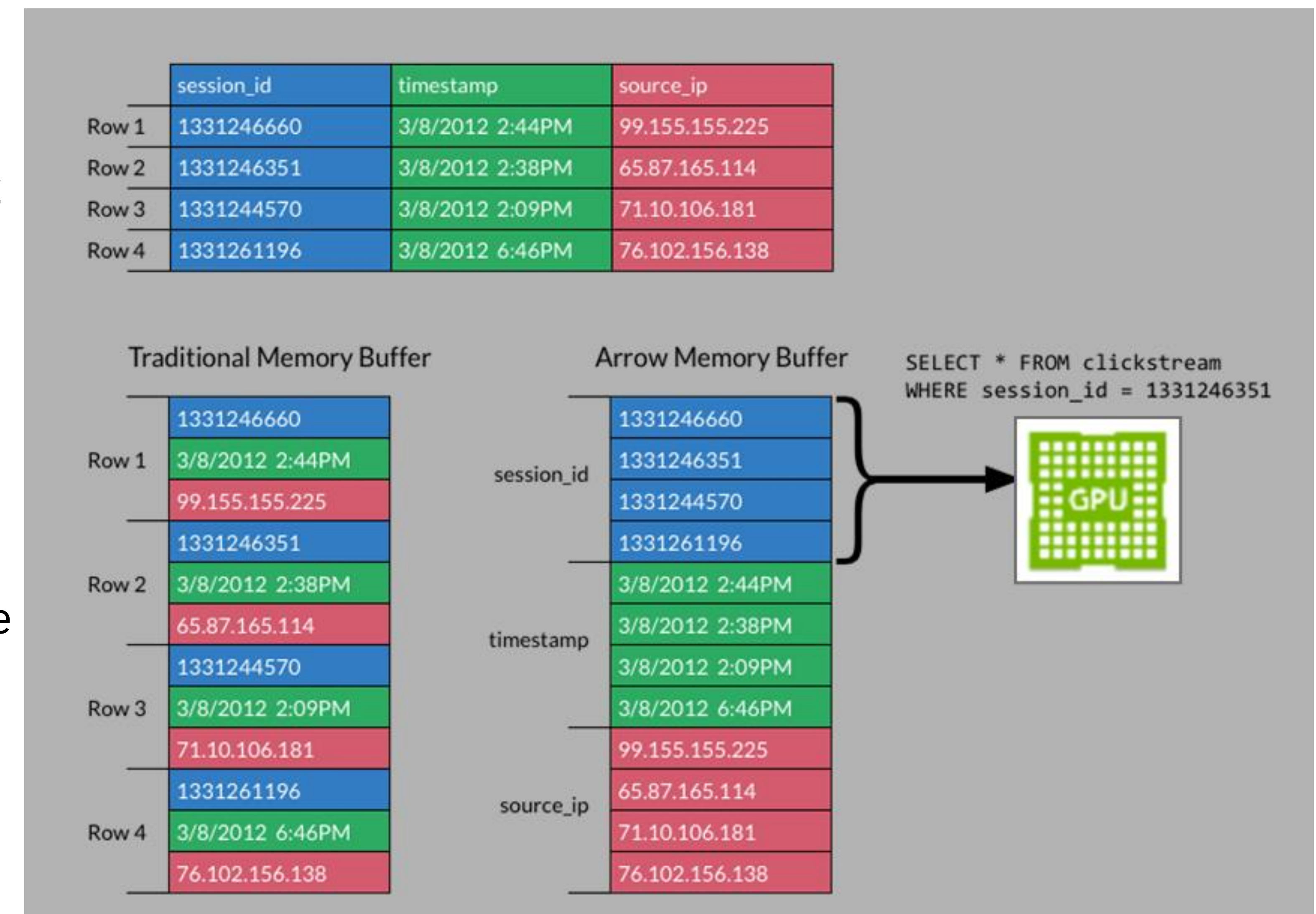
Standardized, language-agnostic in-memory columnar format for structured data



Apache Arrow

Open-source, cross-language development platform for in-memory data processing and analytics

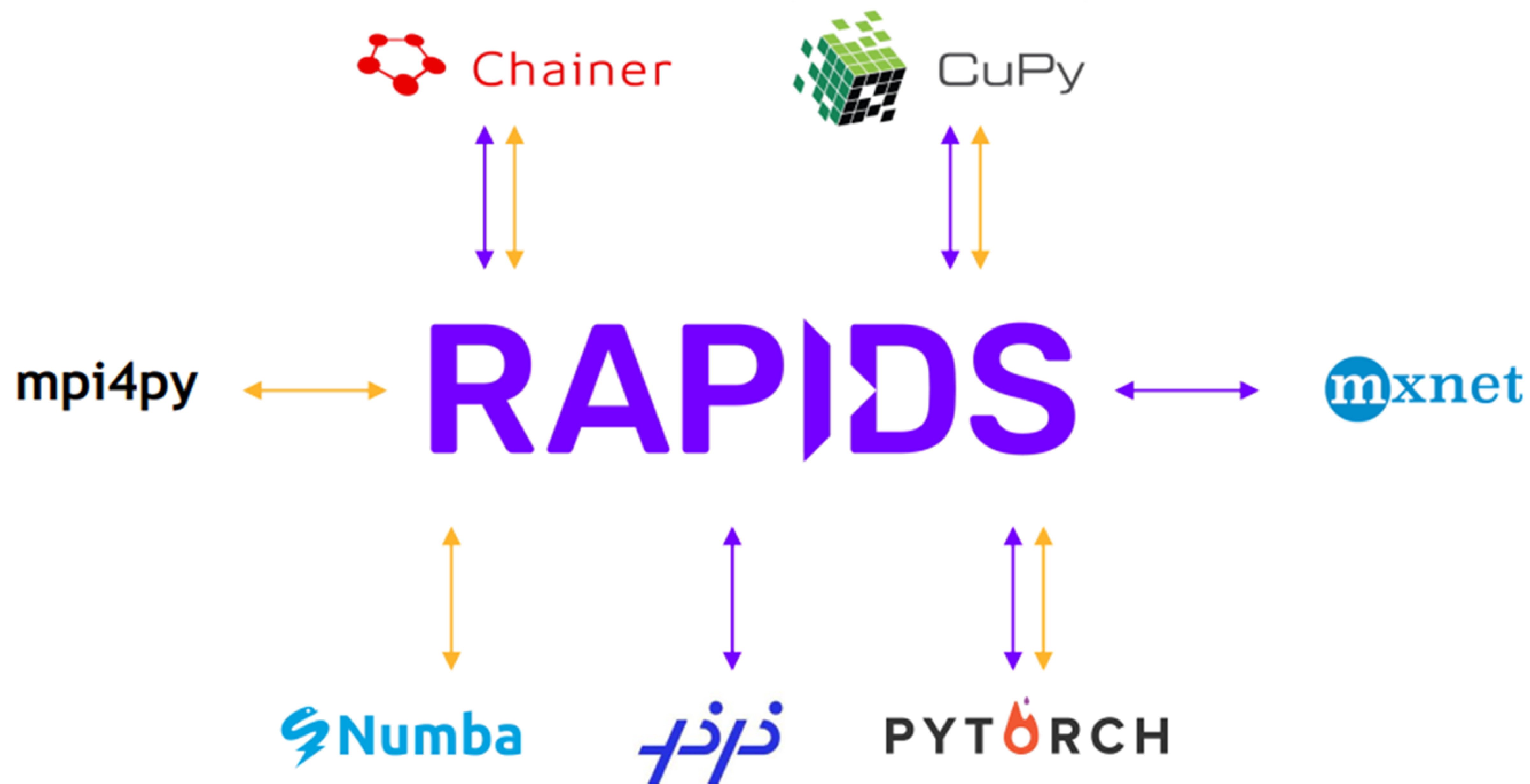
- Standardized In-Memory Columnar Format
 - Data is stored column-by-column instead of row-by-row
 - Enables efficient compression by grouping similar data types together
 - Common data representation that can be shared across different systems and programming languages without copying or transforming
 - Eliminates the need for serialization and deserialization between systems or processes
 - Direct access to data in memory allows for zero-copy reads, reducing memory requirements
 - Columnar format enables vectorized processes using SIMD (single instruction, multiple data) operations
- Integrates well with columnar file formats such as Parquet, allowing for efficient reading of data from disk
- Supports Complex Data Types
- Columnar layout leverages GPU strengths



Interoperability with CuPy

Interoperability of the GPU PyData Ecosystem

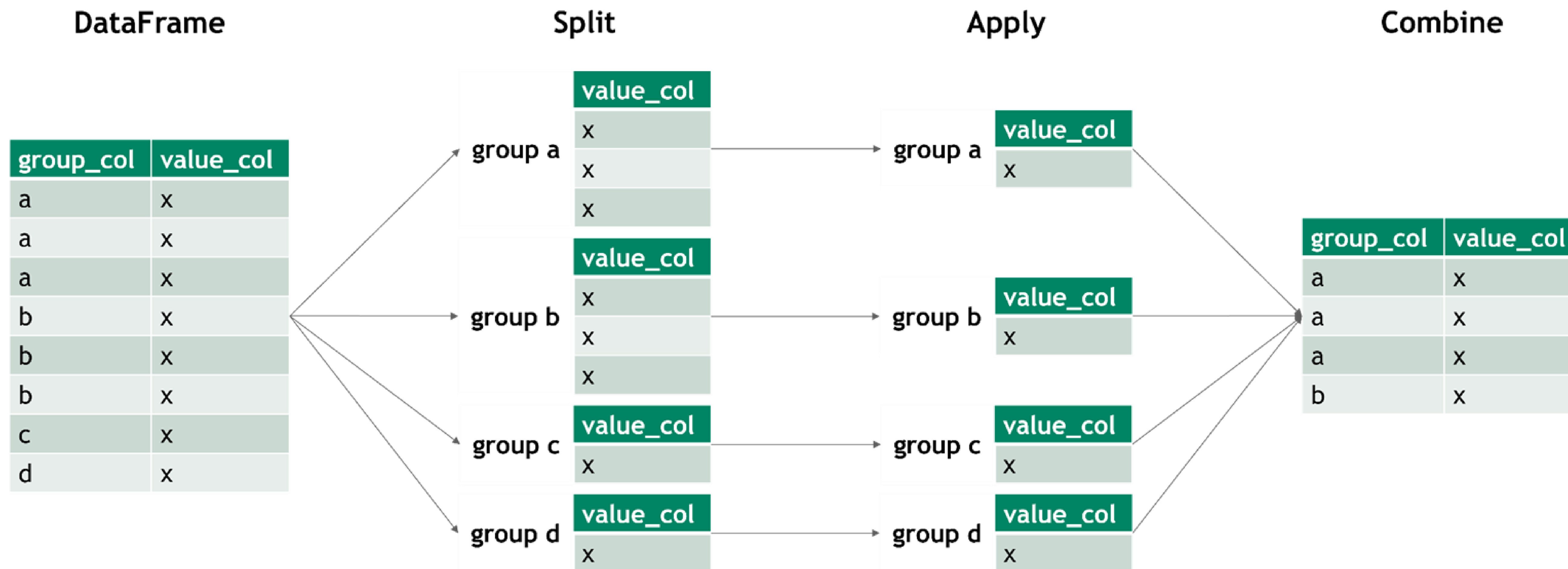
Using `DLPack` and `__cuda_array_interface__` – the CUDA array interface is a standard format that describes a GPU array to allow sharing GPU arrays between different libraries without needing to copy or convert data



Groupby

Split-apply-combine

- Groupby allows you to separate data into groups and apply aggregation function. This helps in summarizing large datasets and extracting meaningful insights.
- DataFrameGroupBy
 - computations / descriptive stats
 - .agg/aggregate - compute a summary statistic (or statistics) for each group
 - .transform - perform some group-specific computations and return a like-indexed object
 - .apply - apply function group-wise and combine the results together



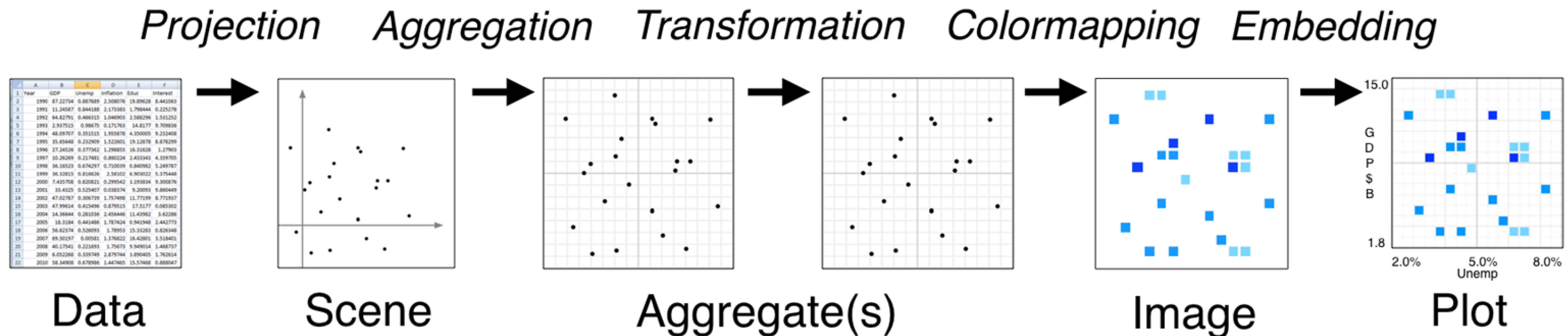
Interactive Visualization

Disruption due to slow processing speed creates friction in the analysis process

- While data visuals are an effective tool for explaining data insights at the end of a project, they should ideally be used throughout the data exploration and enriching process. Visualization excels at enhancing data understanding by finding outliers, anomalies, and patterns not easily surfaced by purely analytical methods.
 - Interactions such as filtering, selecting, or re-rendering points that are slower than 7-10 seconds result in a disruption of a user's short-term memory and train of thought. This disruption creates friction in the analysis process.

Datashader - a graphics pipeline system for creating meaningful representations of large datasets quickly and flexibly

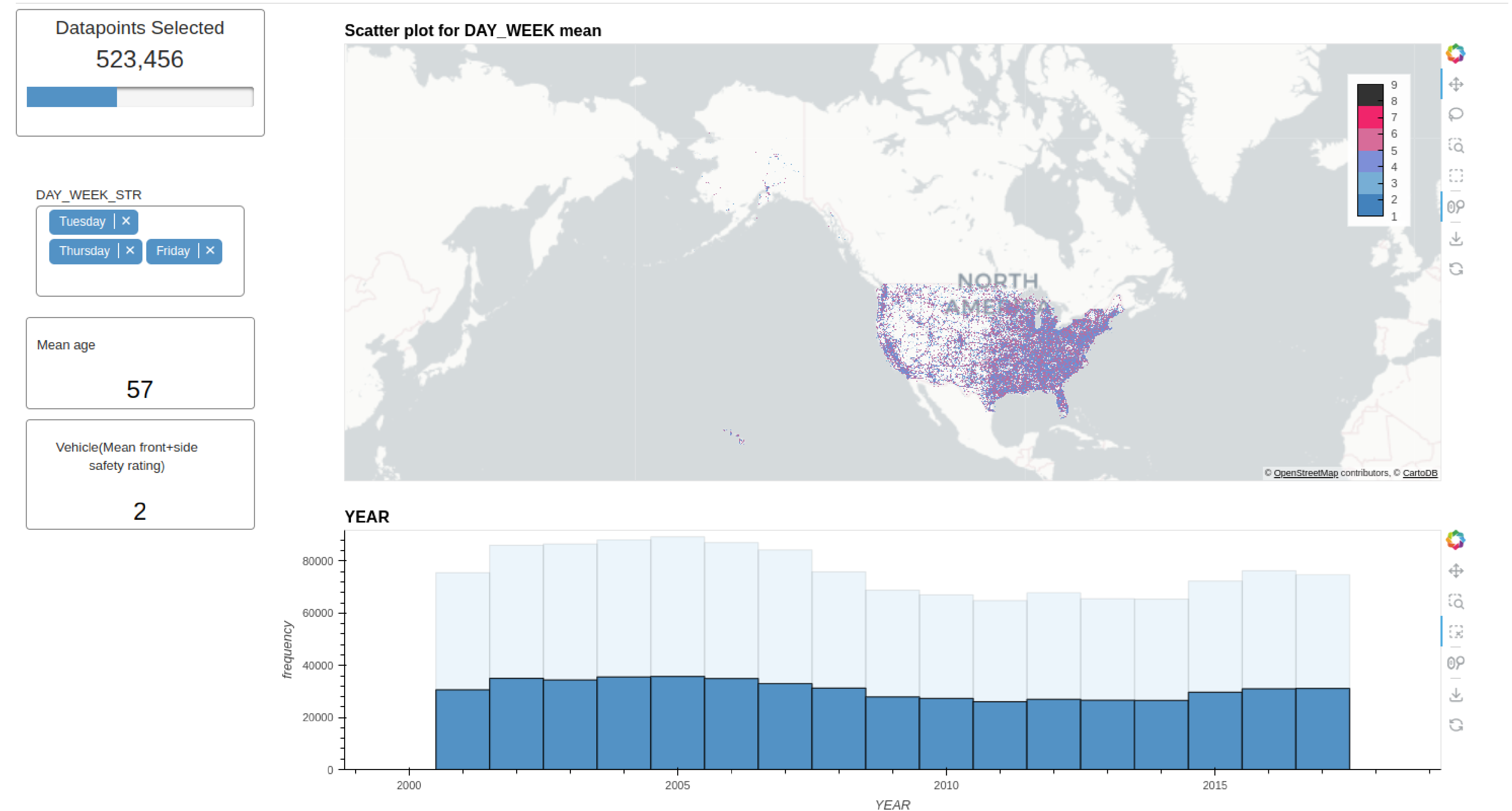
- Work by rasterizing or aggregating large datasets into regular grids that can be viewed as images.



cuxfilter (cu-x-filter)

Cross-filtering apply interactions from one chart as filters to other charts in the dashboard

- Efficient GPU-backed in-browser visualization engine
- Built on a host of pyViz ecosystem tools
- Bokeh and Datashader charts
- Interact with multiple charts with cross-filtering widgets



Polars

- High Performance DataFrame Library written in Rust.
- Gaining rapid adoption in data science community as a Pandas alternative.

Key Differences from existing libraries:

- Significantly faster performance, especially for large datasets
- Leverages all CPU cores with parallel processing
- Build on Apache Arrow columnar memory format
- Lazy Execution & Query Optimizer
- Streaming API allows data to be processed in batches

Lazy Execution and Query Optimization

Lazy Execution

- Polars uses lazy evaluation by default, but has eager execution available
- Queries are not executed immediately, but built into an execution plan

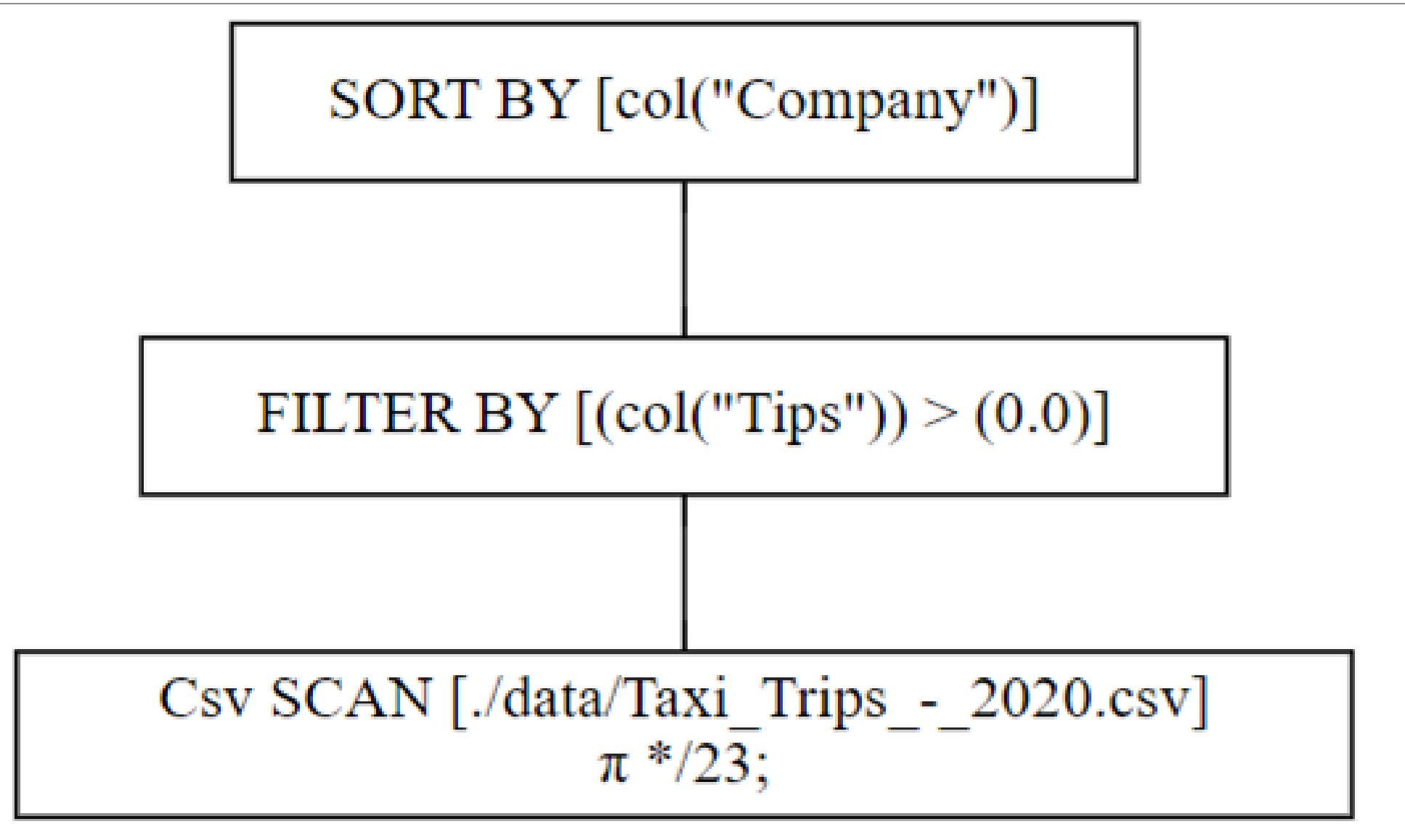
Benefits of lazy evaluation:

- Allows global query optimization
- Minimizes unnecessary computations and memory usage
- Enables out-of-core processing for datasets larger than RAM

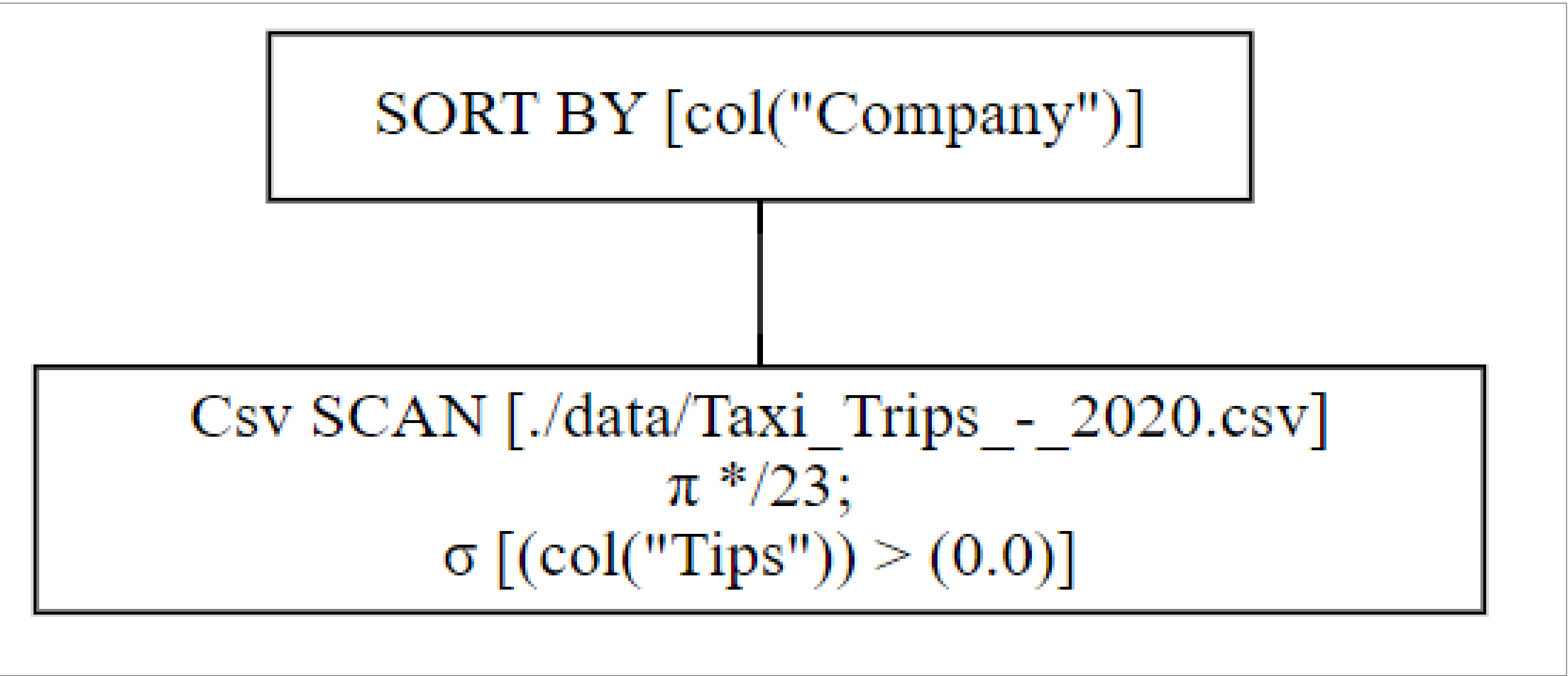
Query optimizer:

- Analyzes full query plan to determine most efficient execution
- Applies optimizations like predicate pushdown and projection pushdown
- Parallelizes operations across CPU cores automatically

Example of Query Optimization



Unoptimized Query



Optimized Query

In this example, Polar’s execution engine decided to filter the column at the same time the csv is being loaded- saving compute power and speeding up runtimes!

Polars + RAPIDS cuDF

- NVIDIA RAPIDS accelerates Polars through a GPU engine built into the lazy API.
- The GPU Engine object can be passed to query execution for automatic acceleration (collect() function).

Execution Process

- Polars builds and optimizes the query plan
- cuDF inspects the plan, replacing supported operations with GPU-accelerated versions
- Unsupported operations fallback to CPU execution with minimal overhead.

Distributed Computing

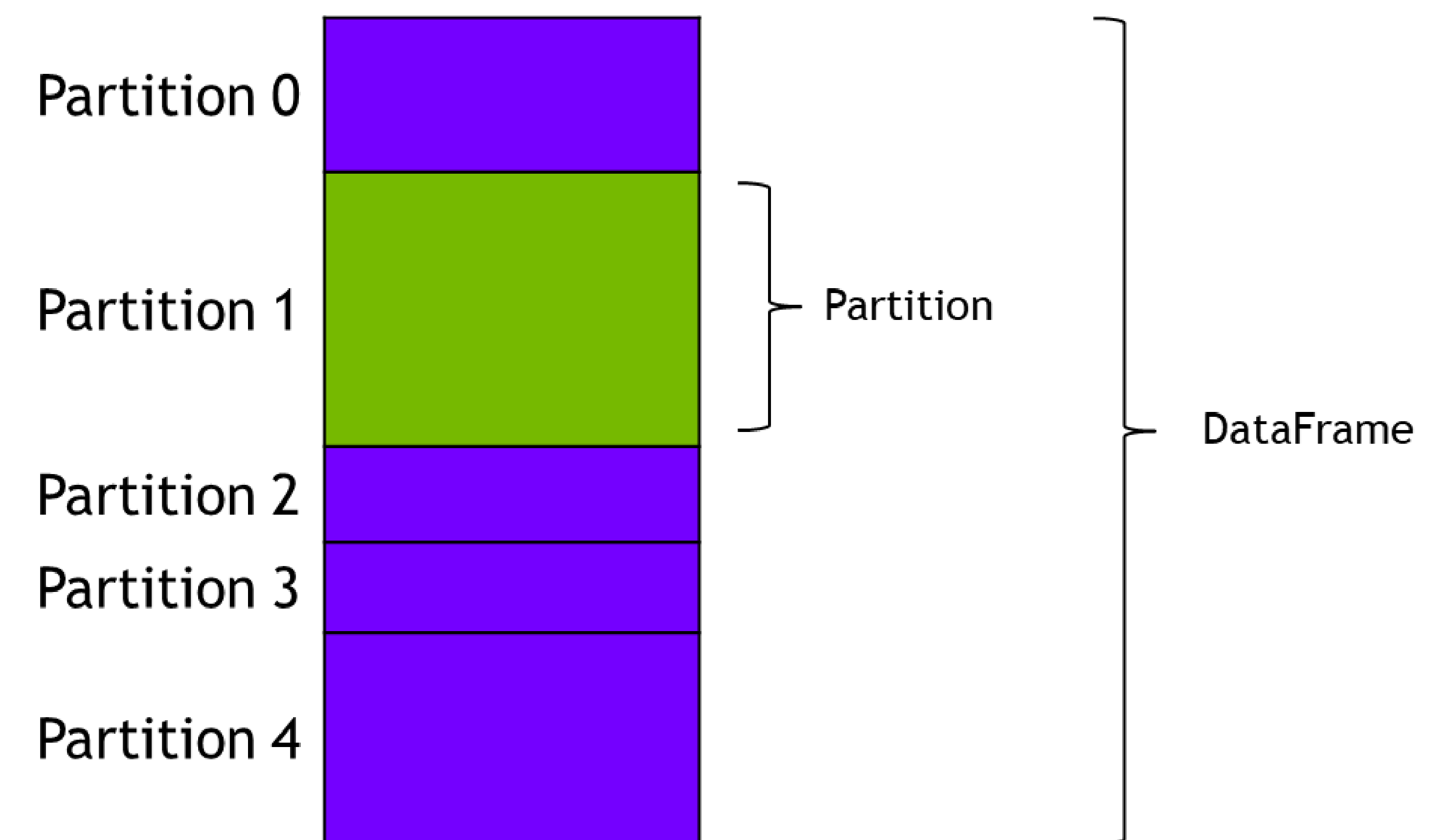
Scaling seamlessly

Distributed computing frameworks:

- Accelerator for Apache Spark
- Distributed computing with Dask
 - Dask-cuDF
 - NVTabular (tightly integrated with Dask-cuDF)

How it works:

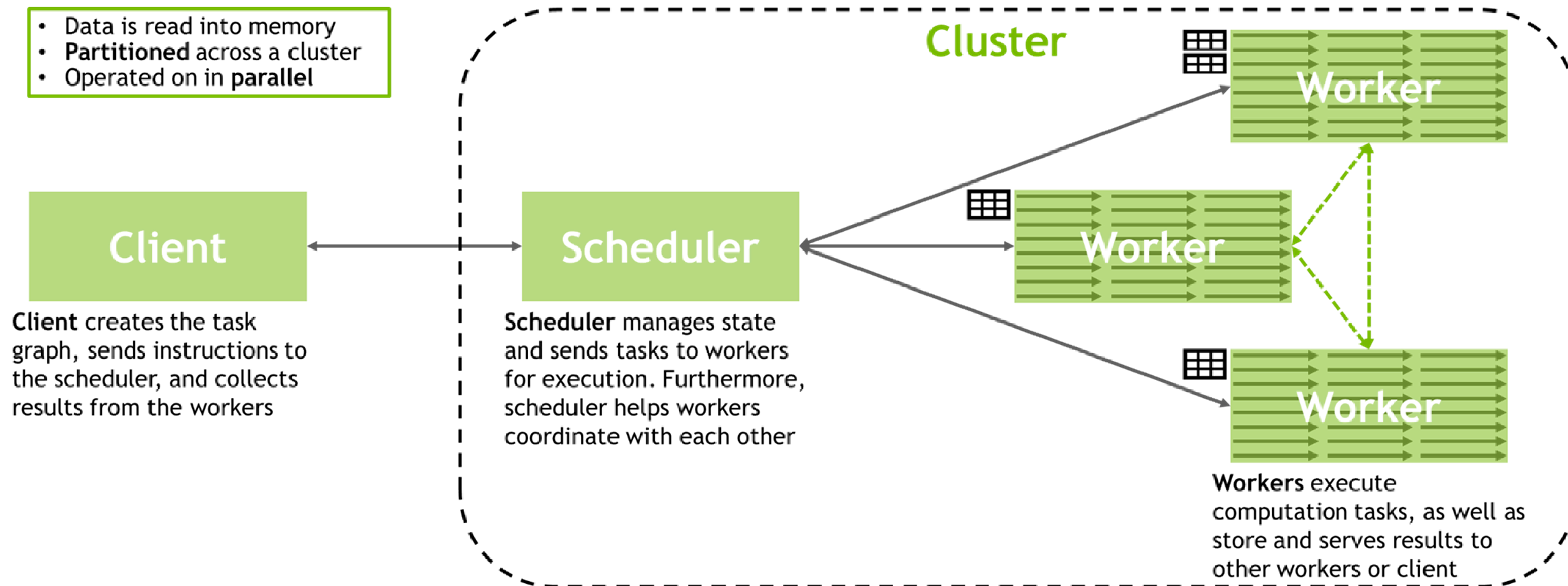
- Data is split into many smaller DataFrames, which we call chunks/partitions
- Uses familiar APIs as those found in the PyData ecosystem.
 - APIs for entire DataFrame object, but methods operate across partitions
- Support for multi-GPU, multi-node
 - Able to handle massive datasets but introduces node-to-node communication
- Important to rebalance data across workers when necessary



RAPIDS + Dask

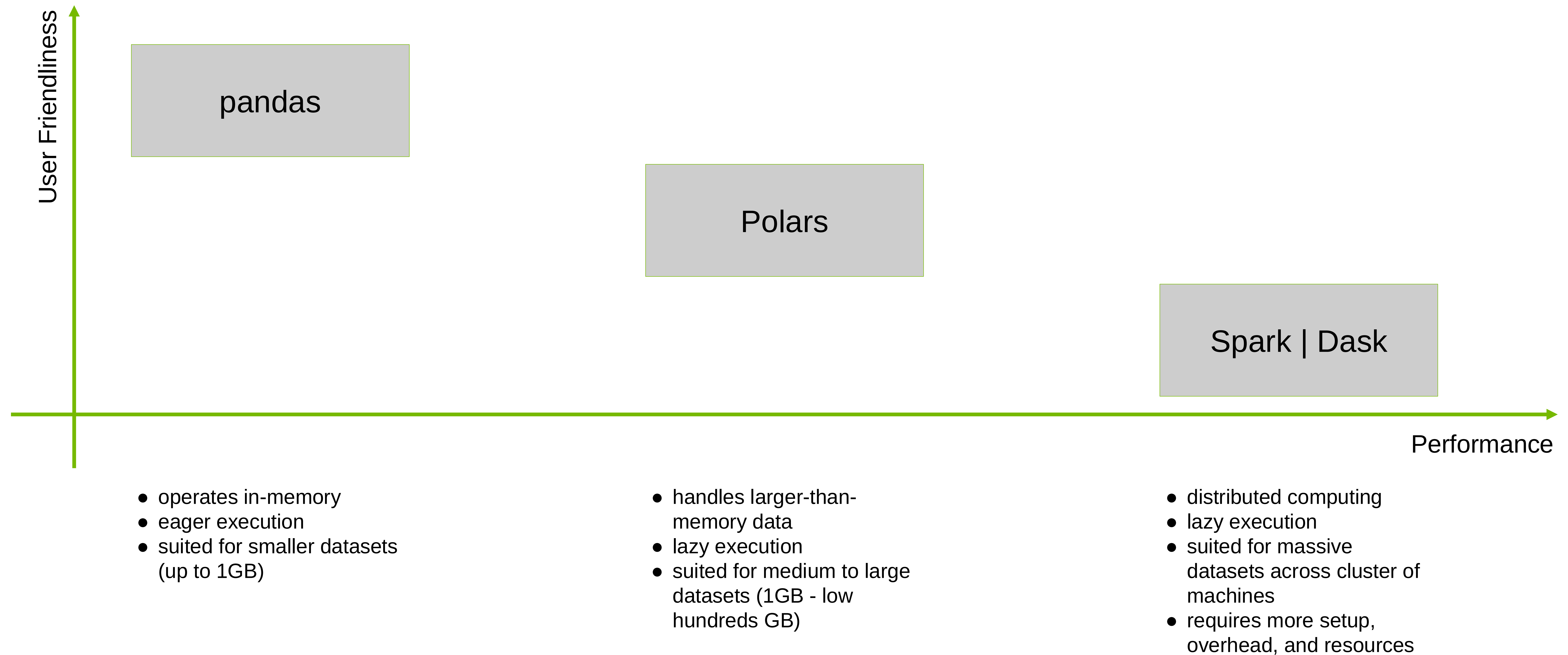
Distributed parallel computing on multi-GPU

- Dask is a parallel computing library built to scale Python workloads from single machines to supercomputer clusters
- Executes operations on chunks of data and aggregates results, thus enabling working with datasets that are larger than memory
- scheduler coordinates the actions of several dask-worker processes spread across multiple machines
- Operations on Dask DataFrame build task graphs, which are optimized prior to execution



Accelerating Data Processing

Choosing tools based on use case, data size, performance requirements, and existing infrastructure



Accelerating Data Processing

Choosing tools based on use case, data size, performance requirements, and existing infrastructure

